

HW 2-B: OWASP Juice Shop / OWASP Top 10

GitHub account: srijapentyala

Part 1: Secure feature design

I ran Juice Shop locally and used three bugs as the spec for registration. A new account should not be able to rewrite a query, plant a script, or sign in with a guessable password.

Login concatenates the email into SQL. I entered ' OR 1=1-- with a junk password and was signed in as admin@juice-sh.op. Juice Shop awarded Login Admin. The same mistake on a registration insert would let the email close the statement and write an administrator row. A parameterized query keeps the email as a bound value, so it cannot change the statement.

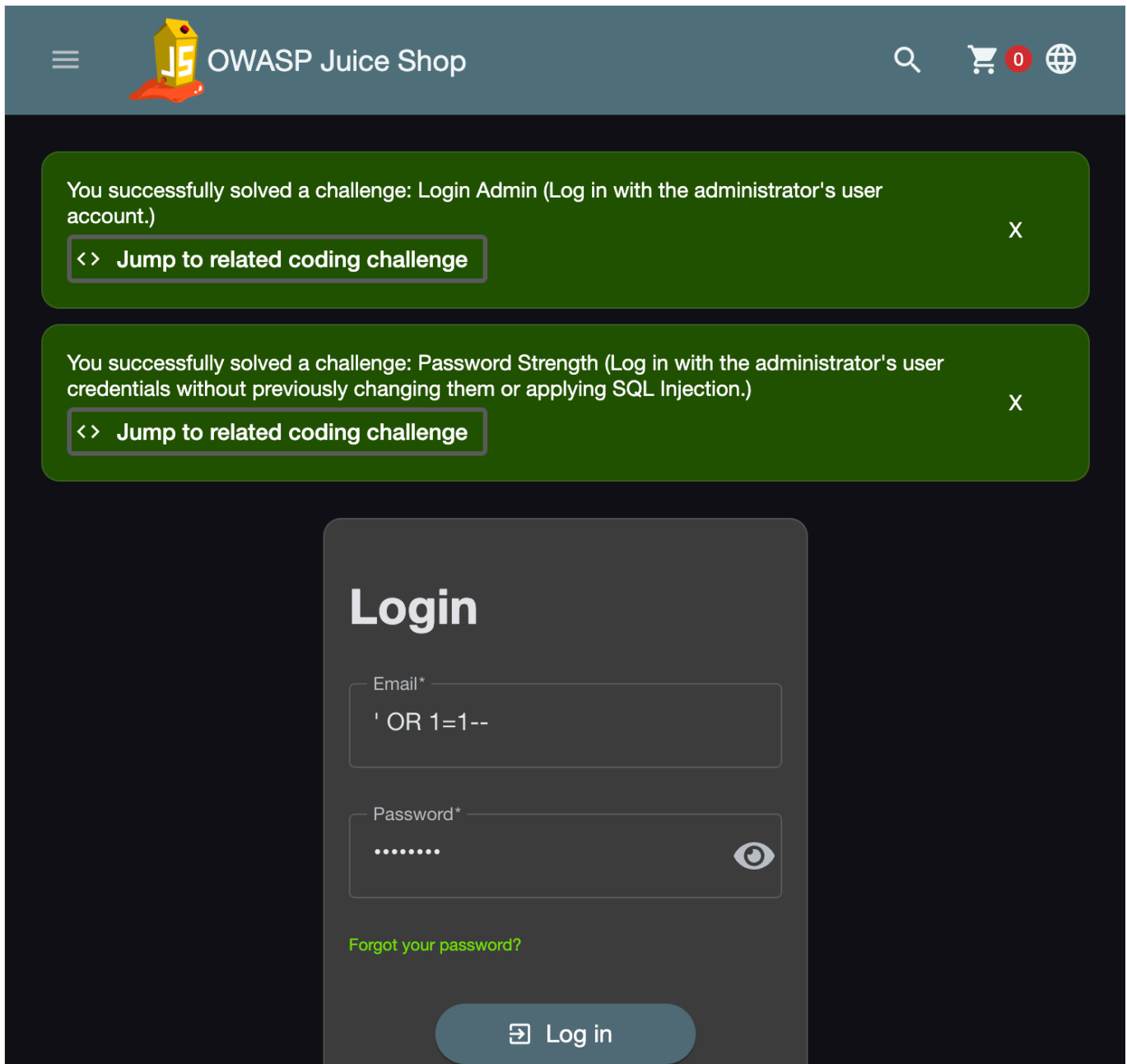


Figure 1. The injected email is still in the form. Juice Shop awarded Login Admin for that login, and Password Strength for the real administrator password admin123.

Search inserts the query as HTML. My term rendered as a yellow XSS heading, not as text. A username saved at registration would do the same thing wherever the profile is shown. Encode that value on output, and send a Content-Security-Policy that blocks inline script, so a tag in the name is displayed and does not run.

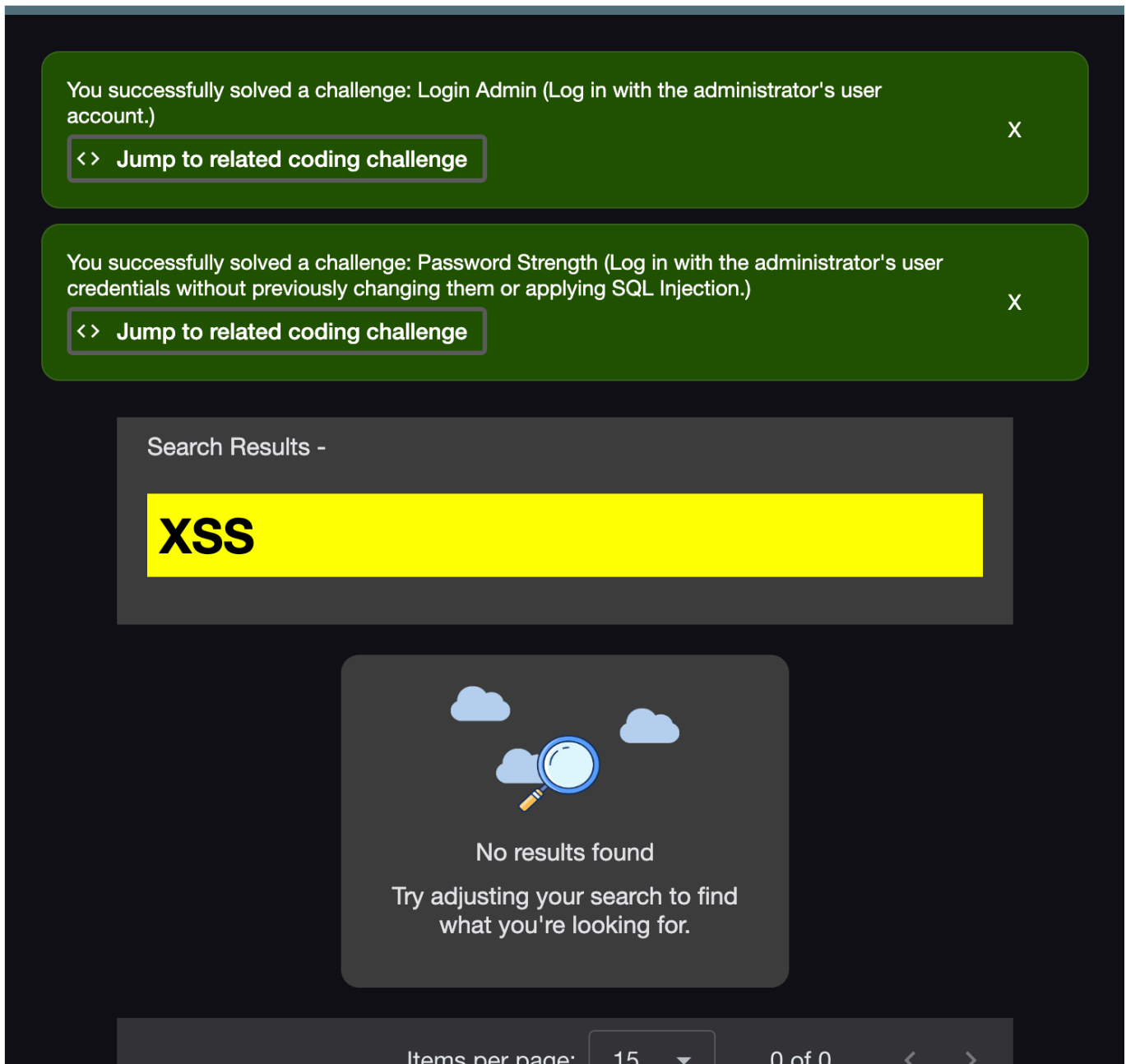


Figure 2. The search term was parsed as HTML. The tags never appear as text.

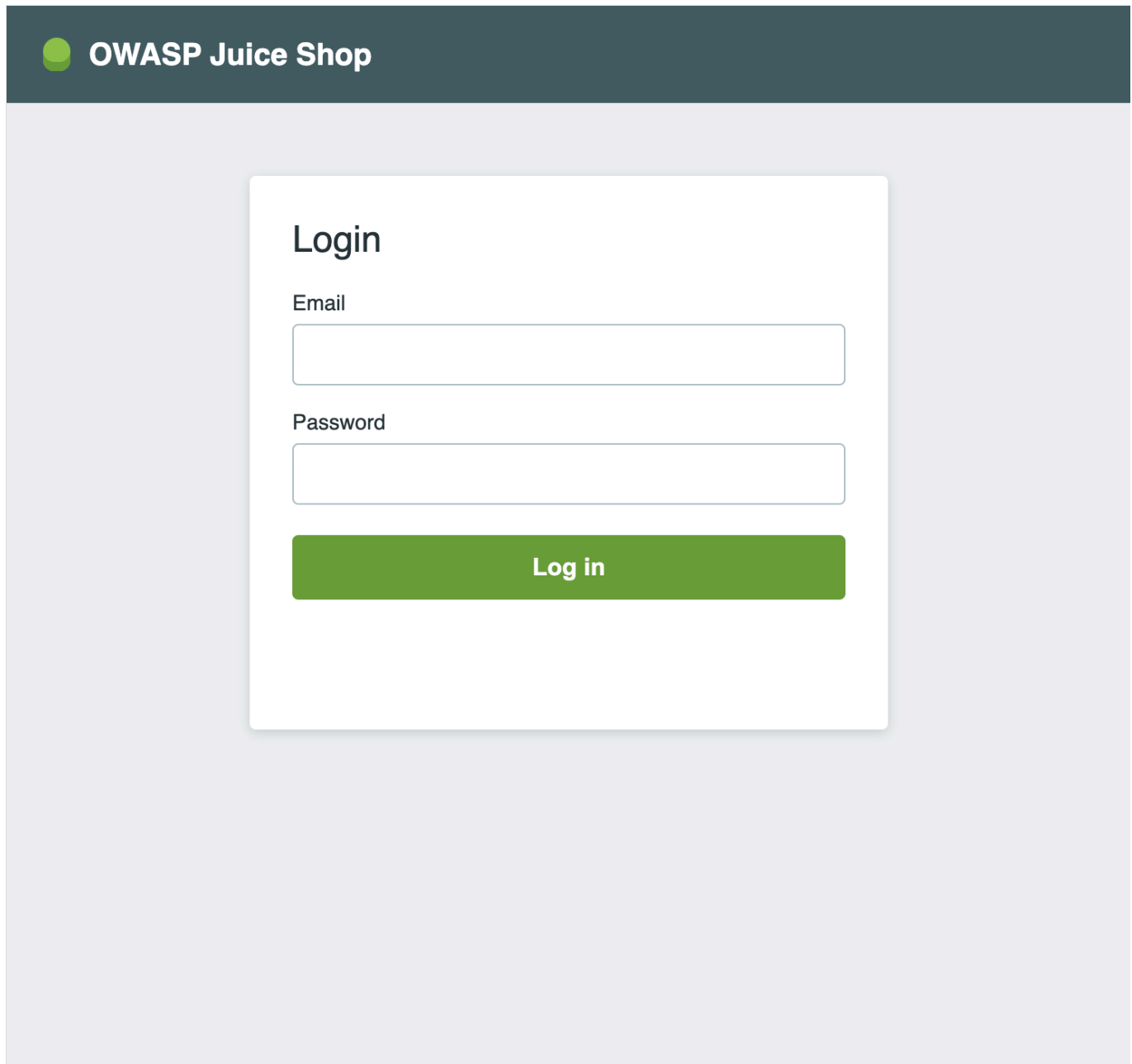
The administrator password is admin123, and the login token includes its MD5 hash. Password Strength solved only when I used that password, not injection. Registration should reject short and common passwords, store only a bcrypt hash with a unique salt and a cost of 12, and never send the hash back to the browser.

```
bcrypt.hashpw(password.encode(), bcrypt.gensalt(rounds=12))  
bcrypt.checkpw(password.encode(), stored_hash)
```

Part 2: Front-end login form

I built a login page with the same two fields as Juice Shop: email and password. login.js blocks an empty submit, requires “@” in the email, and requires a password of at least 8 characters. Those three errors stay in the browser. server.py repeats them on POST /login, then compares the password with a PBKDF2-HMAC-SHA256 hash of the demo account using a constant-time check. A wrong password and an unknown email return the same sentence. Input is not placed in a SQL string. Run `python3 server.py` and open `http://127.0.0.1:8080`. The demo account is `demo@juice-sh.op` / `JuiceShop1`. The README in the repository explains the same steps.

Repository: <https://github.com/srijapentyala/hw2b-juice-shop-login>



The image shows a web browser window displaying the OWASP Juice Shop login page. At the top, there is a dark blue header with the OWASP Juice Shop logo (a green circle) and the text "OWASP Juice Shop". Below the header, the main content area is light gray. In the center, there is a white login form with a green border. The form has a title "Login" in bold black text. Below the title, there are two input fields: "Email" and "Password". The "Email" field is a white rectangle with a thin blue border. The "Password" field is a white rectangle with a thin blue border. Below the password field, there is a green button with the text "Log in" in white. The form is centered on the page.

Figure 3. Login form before submission.

Login

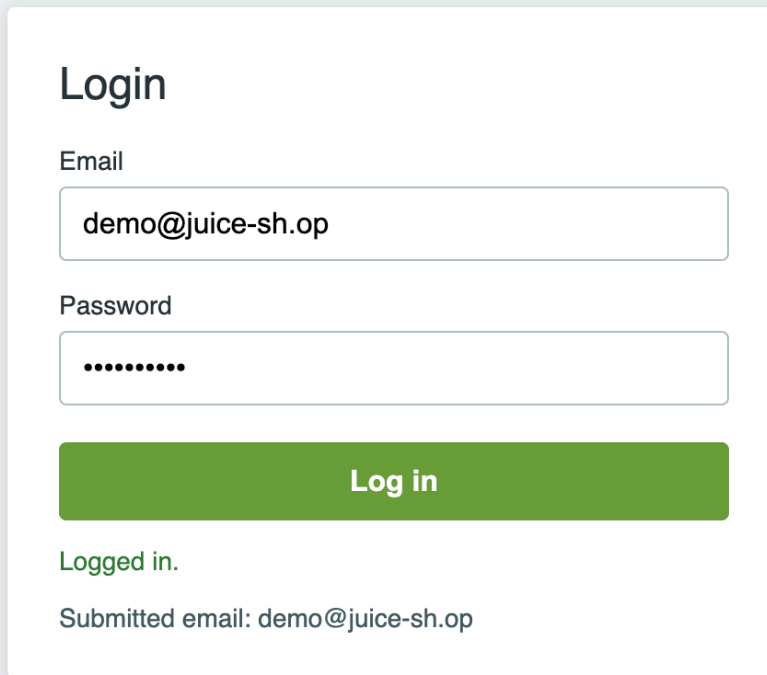
Email

Password

Log in

Email and password are required.

Figure 4. Client-side check rejects an empty submission.



Login

Email

Password

Log in

Logged in.

Submitted email: demo@juice-sh.op

Figure 5. Demo account passes both the browser check and the server check.

Part 3: Trying to break the form

I attacked `http://127.0.0.1:8080` twice, using the same ideas that worked on Juice Shop.

1. SQL injection. I started the server, opened the page, entered ' `OR 1=1--@juice-sh.op` as the email and `password1` as the password, and clicked Log in. The "@" and length checks passed, so the request reached the server. It answered "Invalid email or password." There is no SQL string to rewrite, so the login failed.

Login

Email

Password

Log in

Invalid email or password.

Submitted email: ' OR 1=1--@juice-sh.op

Figure 6. The SQL fragment is stored as an email address. Login fails.

2. XSS, which succeeds on the code in the repository. In `login.js` the result line is assigned with `innerHTML`. I entered `<img src=x onerror="document.getElementById('message').textContent='JavaScript executed'">@a.co` and password `password1`, then clicked Log in. The image could not load, so the handler ran and replaced the status text with “JavaScript executed.” The login itself still failed. Only the display of the email was unsafe. The same steps work after `python3 server.py` at `http://127.0.0.1:8080`.

Login

Email

Password

Log in

JavaScript executed

Submitted email: @a.co

Figure 7. The status line was written by the image handler, not by the login code.

Fix: assign that line with `textContent`, so the email is shown as characters, and send `Content-Security-Policy: script-src 'self'` so an inline handler cannot run even if a later edit inserts one.